

Personal Finance & Money Flow Tracker

Technical Design Document (TDD)

Document Version: 0.2

Status: Initial Technical Design — Subject to Refinement

Purpose: Implementation starting point

Application Phase: Phase 1

Future Phase: Phase 2 Dashboard & Reporting, Phase 3 Authentication & User-Level Access

1. Document Purpose

This Technical Design Document defines the initial technical architecture, application components, Google Sheets data model, API design, frontend behavior, transaction processing logic, validation, deployment model, and implementation approach for the Personal Finance & Money Flow Tracker.

This document is an **implementation baseline** and will be refined during development before the design is considered final.

2. Business Concept

The application is a lightweight personal money-flow tracking system.

The system records exactly three transaction types:

1. Expense
2. Income
3. Transfer

The system supports exactly three formal account types:

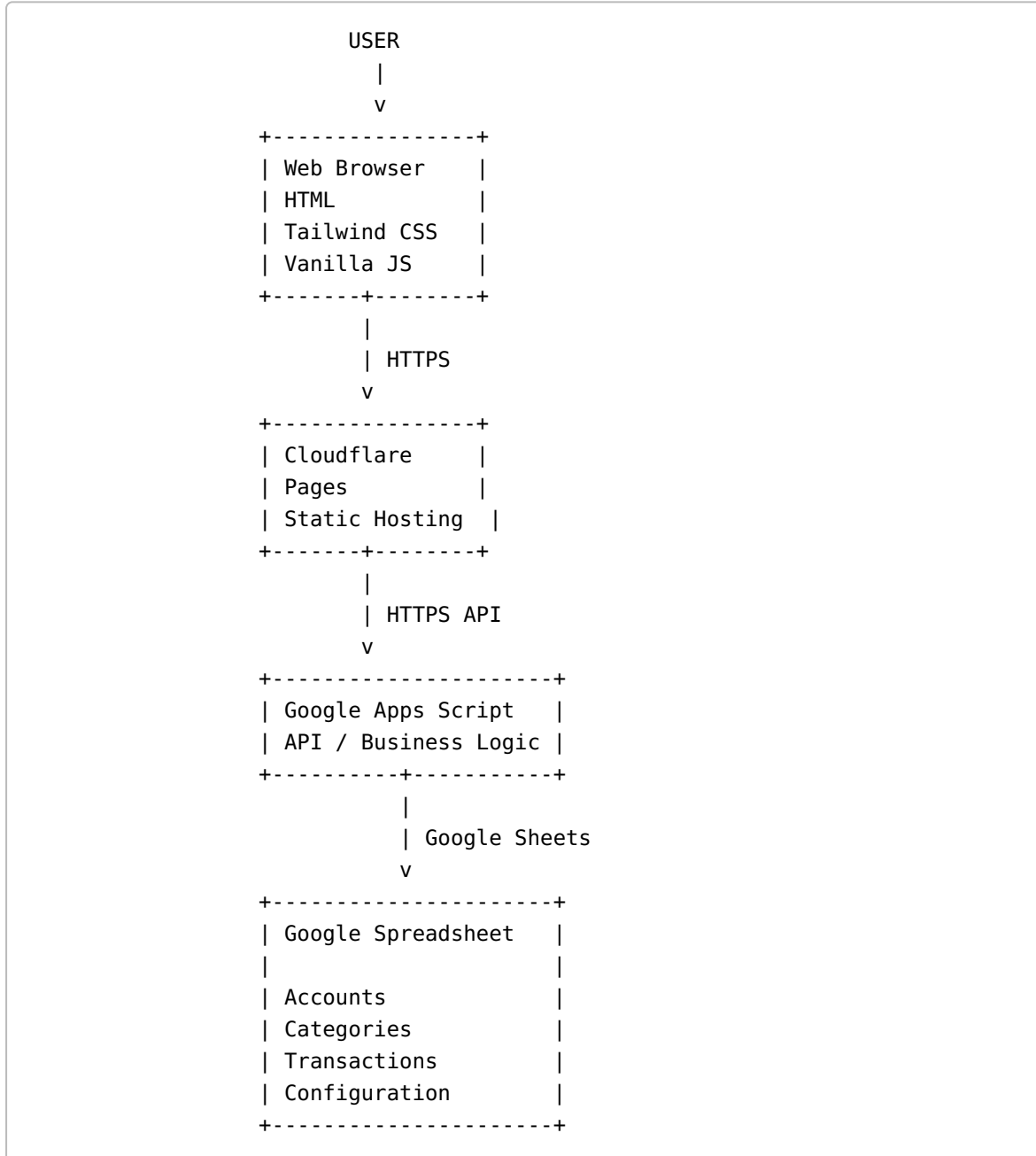
1. Bank Account
2. Credit Card
3. Gift Card

The following are not separate transaction types:

- Credit Card Payment
- Gift Card Purchase
- Cash Withdrawal
- Cash Deposit
- Investment
- Loan Payment
- Interest
- Cashback

These scenarios are represented using Expense, Income, or Transfer according to the business rules.

3. High-Level Technical Architecture



4. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5	Application structure

Layer	Technology	Purpose
Styling	Tailwind CSS	Responsive UI styling
Frontend Logic	Vanilla JavaScript	Application behavior
Icons	Lucide Icons	UI icons
Charts	Chart.js	Phase 2 reporting
Static Hosting	Cloudflare Pages	Frontend hosting
Backend/API	Google Apps Script	API and business logic
Database	Google Sheets	Data persistence
HTTP Communication	Fetch API	Frontend/API communication
Data Format	JSON	API request/response format
Version Control	Git	Source control
Repository	GitHub	Code repository
API Testing	Postman / Bruno	API testing
Browser Testing	Google Chrome / DevTools	UI testing and debugging
Documentation	Markdown	Technical documentation

5. Architecture Responsibilities

5.1 Frontend

The frontend is responsible for:

- displaying application UI
- displaying transaction form
- dynamically changing form fields
- loading master data
- submitting transactions
- displaying transactions
- editing transactions
- deleting transactions
- displaying validation messages
- displaying API errors
- formatting dates and amounts

The frontend is **not responsible for authoritative financial calculations.**

5.2 Google Apps Script

Google Apps Script acts as the backend/API layer.

Responsibilities:

- receive API requests
 - validate requests
 - validate account references
 - validate category references
 - process transactions
 - generate transaction IDs
 - read/write Google Sheets
 - calculate balances
 - return JSON responses
 - handle backend errors
-

5.3 Google Sheets

Google Sheets is the Phase 1 persistent data store.

It contains:

```
Accounts
Categories
Transactions
Configuration
```

The spreadsheet is the authoritative source of persistent application data.

6. Google Spreadsheet Structure

The application will initially use one Google Spreadsheet.

```
PersonalFinanceTracker
|
+-- Accounts
+-- Categories
+-- Transactions
+-- Configuration
```

7. Accounts Sheet

The `Accounts` sheet contains master data for:

- Bank Accounts
- Credit Cards
- Gift Cards

7.1 Accounts Columns

Column	Column Name	Description
A	Account_ID	Unique account identifier
B	Account_Name	Account display name
C	Account_Type	BANK / CREDIT_CARD / GIFT_CARD
D	Opening_Balance	Initial balance/value

Final Accounts Schema

```
Account_ID
Account_Name
Account_Type
Opening_Balance
```

No additional fields are required at this stage.

The following fields are intentionally **not included**:

```
Currency
Active
Created_Date
Updated_Date
```

8. Account Type Values

`Account_Type` supports only:

```
BANK
CREDIT_CARD
GIFT_CARD
```

Example:

Account_ID	Account_Name	Account_Type	Opening_Balance
ACC001	SBI Bank	BANK	100000
ACC002	HDFC Bank	BANK	50000
ACC003	ICICI Credit Card	CREDIT_CARD	0
ACC004	Amazon Gift Card	GIFT_CARD	5000

9. Categories Sheet

The **Categories** sheet contains the master category list.

No subcategories are required.

9.1 Categories Columns

Column	Column Name	Description
A	Category_ID	Unique category identifier
B	Category_Name	Category display name

Final Categories Schema

Category_ID
Category_Name

The following fields are intentionally **not included**:

Active
Created_Date
Updated_Date

10. Categories Example

Category_ID	Category_Name
CAT001	Food
CAT002	Travel
CAT003	Shopping
CAT004	Investment

Category_ID	Category_Name
CAT005	Loan Payment

The exact category list is maintained in the master sheet.

The frontend must retrieve categories dynamically and must not hard-code them.

11. Transactions Sheet

The **Transactions** sheet is the primary financial ledger.

11.1 Transaction Columns

Column	Column Name	Description
A	Transaction_ID	Unique transaction identifier
B	Transaction_Date	Business transaction date
C	Transaction_Type	EXPENSE / INCOME / TRANSFER
D	Amount	Transaction amount
E	Category_ID	Category reference
F	Payment_Method	BANK / CREDIT_CARD / GIFT_CARD
G	Paid_From_Account_ID	Expense source account
H	Received_Into_Account_ID	Income destination account
I	From_Account_ID	Transfer source
J	To_Account_ID	Transfer destination
K	Notes	Optional notes
L	Created_Date	Technical record creation timestamp
M	Updated_Date	Technical record update timestamp
N	Status	ACTIVE / DELETED

12. Transaction Date Format

The application uses:

DD-MM-YYYY

for displaying and entering transaction dates.

Example:

07-09-2026

The business transaction date does not contain a time.

Technical timestamps such as `Created_Date` and `Updated_Date` are separate system metadata and are not displayed as transaction dates.

13. Transaction Date Storage

The application should maintain a consistent internal date representation.

Recommended approach:

UI:
DD-MM-YYYY

API:
DD-MM-YYYY

Google Sheet:
Date value formatted as DD-MM-YYYY

The implementation should ensure that browser, Apps Script, and Google Sheets do not introduce unexpected timezone-related date shifts.

14. Transaction Type

Only three transaction types are allowed:

EXPENSE
INCOME
TRANSFER

The backend must reject unsupported transaction types.

15. Expense Transaction Schema

For an Expense:

```
Transaction_ID
Transaction_Date
Transaction_Type
Amount
Category_ID
Payment_Method
Paid_From_Account_ID
Notes
```

Unused fields:

```
Received_Into_Account_ID
From_Account_ID
To_Account_ID
```

16. Income Transaction Schema

For Income:

```
Transaction_ID
Transaction_Date
Transaction_Type
Amount
Category_ID
Received_Into_Account_ID
Notes
```

17. Transfer Transaction Schema

For Transfer:

```
Transaction_ID
Transaction_Date
Transaction_Type
Amount
From_Account_ID
```

To_Account_ID
Notes

Category and payment method are not required for Transfer.

18. Configuration Sheet

The **Configuration** sheet stores application-level configuration.

Columns

Column	Name	Description
A	Config_Key	Configuration key
B	Config_Value	Configuration value
C	Description	Configuration description

Example:

Config_Key	Config_Value	Description
APP_NAME	Personal Finance Tracker	Application name
DATE_FORMAT	DD-MM-YYYY	Application date format

19. Transaction ID

Transaction IDs are generated by Google Apps Script.

Recommended format:

TXN-YYYYMMDD-XXXXXX

Example:

TXN-20260907-A81F32

The frontend does not generate the authoritative transaction ID.

20. Account ID

Account IDs are master-data identifiers.

Recommended format:

```
ACC001
ACC002
ACC003
```

Transactions reference accounts using `Account_ID`.

21. Category ID

Category IDs are master-data identifiers.

Recommended format:

```
CAT001
CAT002
CAT003
```

Transactions reference categories using `Category_ID`.

22. API Architecture

Google Apps Script will be deployed as a Web App.

The frontend communicates with the Web App using HTTPS.

Conceptually:

```
Browser
|
| HTTP Request
v
Google Apps Script
|
v
Google Sheets
```

23. API Request Structure

Recommended request format:

```
{
  "action": "CREATE_TRANSACTION",
  "data": {
    "transactionDate": "07-09-2026",
    "transactionType": "EXPENSE",
    "amount": 2500,
    "categoryId": "CAT001",
    "paymentMethod": "BANK",
    "paidFromAccountId": "ACC001",
    "notes": "Dinner"
  }
}
```

24. Standard API Response

Successful response:

```
{
  "success": true,
  "message": "Transaction created successfully",
  "data": {
    "transactionId": "TXN-20260907-A81F32"
  }
}
```

Error response:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Amount must be greater than zero"
  }
}
```

25. API Operations

Initial API operations:

```
GET_MASTER_DATA
GET_TRANSACTIONS
GET_TRANSACTION
CREATE_TRANSACTION
UPDATE_TRANSACTION
DELETE_TRANSACTION
GET_BALANCES
```

26. GET_MASTER_DATA

Returns:

```
Accounts
Categories
Configuration
```

Example:

```
{
  "success": true,
  "data": {
    "accounts": [],
    "categories": [],
    "configuration": {}
  }
}
```

The frontend calls this during application initialization.

27. GET_TRANSACTIONS

Returns active transaction records.

Phase 1 can initially return all active transactions.

Example:

```
{
  "success": true,
  "data": {
    "transactions": []
  }
}
```

```
}  
}
```

28. GET_TRANSACTION

Retrieves a specific transaction using `Transaction_ID`.

Example request:

```
{  
  "action": "GET_TRANSACTION",  
  "transactionId": "TXN-20260907-A81F32"  
}
```

29. CREATE_TRANSACTION

Creation flow:

```
Frontend  
  |  
  v  
Client-side validation  
  |  
  v  
API Request  
  |  
  v  
Apps Script  
  |  
  +--> Validate Transaction Type  
  |  
  +--> Validate Amount  
  |  
  +--> Validate Date  
  |  
  +--> Validate Category  
  |  
  +--> Validate Account  
  |  
  +--> Validate Transaction-specific fields  
  |  
  v  
Generate Transaction ID  
  |
```

```
    v
Append transaction row
    |
    v
Return response
```

30. UPDATE_TRANSACTION

When a transaction is edited:

```
Existing Transaction
    |
    v
Load transaction
    |
    v
Populate form
    |
    v
User changes values
    |
    v
Validate
    |
    v
Update transaction row
    |
    v
Recalculate balances
    |
    v
Refresh UI
```

No additional balancing transaction should be generated.

The edited transaction itself remains the source of the financial impact.

31. DELETE_TRANSACTION

Phase 1 should use logical deletion.

Instead of physically removing the row:

Status = DELETED

Only records with:

Status = ACTIVE

are included in transaction queries and balance calculations.

This provides safer deletion and basic auditability.

32. Transaction Form

The application uses one transaction form for all transaction types.

The form dynamically changes based on:

Transaction Type

Values:

Expense
Income
Transfer

33. Expense Form

Fields:

Transaction Type
Date
Amount
Category
Payment Method
Paid From
Notes

34. Income Form

Fields:

Transaction Type
Date
Amount
Category
Received Into
Notes

35. Transfer Form

Fields:

Transaction Type
Date
Amount
From
To
Notes

36. Dynamic Form Behavior

Expense

Show:

Category
Payment Method
Paid From

Hide:

Received Into
From
To

Income

Show:

Category
Received Into

Hide:

Payment Method
Paid From
From
To

Transfer

Show:

From
To

Hide:

Category
Payment Method
Paid From
Received Into

37. Payment Method

For Expense:

BANK
CREDIT_CARD
GIFT_CARD

When Payment Method is selected, the **Paid From** dropdown is filtered according to account type.

38. Payment Method Mapping

BANK

↓

Accounts where Account_Type = BANK

CREDIT_CARD

↓

Accounts where Account_Type = CREDIT_CARD

GIFT_CARD

↓

Accounts where Account_Type = GIFT_CARD

Example:

Payment Method = CREDIT_CARD

Paid From:

- ICICI Credit Card
- HDFC Credit Card

39. Expense Processing

Example:

SBI Bank

|

| ₹2,000

v

Merchant

Transaction:

Transaction_Type = EXPENSE

Amount = 2000

Payment_Method = BANK

Paid_From_Account_ID = ACC001

Impact:

Bank Balance - ₹2,000
Expense + ₹2,000

40. Income Processing

Example:

Employer
|
| ₹100,000
v
SBI Bank

Transaction:

Transaction_Type = INCOME
Amount = 100000
Received_Into_Account_ID = ACC001

Impact:

Bank Balance + ₹100,000
Income + ₹100,000

41. Transfer Processing

Example:

SBI Bank
|
| ₹20,000
v
HDFC Bank

Transaction:

Transaction_Type = TRANSFER
Amount = 20000

```
From_Account_ID = ACC001
To_Account_ID = ACC002
```

Impact:

```
SBI Bank      - ₹20,000
HDFC Bank     + ₹20,000
Expense       ₹0
Income        ₹0
```

42. Credit Card Expense

Example:

```
ICICI Credit Card
    |
    | ₹10,000
    v
Hotel
```

Transaction:

```
Transaction_Type = EXPENSE
Amount = 10000
Payment_Method = CREDIT_CARD
Paid_From_Account_ID = ACC003
```

Impact:

```
Expense                +₹10,000
Credit Card Outstanding +₹10,000
```

43. Credit Card Bill Payment

Example:

```
SBI Bank
    |
    | ₹10,000
```

v
ICICI Credit Card

This is:

Transaction_Type = TRANSFER

Impact:

SBI Bank	-₹10,000
Credit Card Outstanding	-₹10,000
Expense	₹0

The payment must not create another expense.

44. Gift Card Purchase

Example:

ICICI Credit Card
|
| ₹5,000
v
Amazon Gift Card

Transaction:

Transaction_Type = TRANSFER
Amount = 5000
From_Account_ID = ICICI Credit Card
To_Account_ID = Amazon Gift Card

Impact:

Credit Card Outstanding	+₹5,000
Gift Card Balance	+₹5,000
Expense	₹0

45. Gift Card Usage

Example:

```
Amazon Gift Card
  |
  | ₹1,000
  v
Merchant
```

Transaction:

```
Transaction_Type = EXPENSE
Amount = 1000
Payment_Method = GIFT_CARD
Paid_From_Account_ID = Amazon Gift Card
```

Impact:

```
Gift Card Balance -₹1,000
Expense           +₹1,000
```

46. Cash Withdrawal

Cash is not a formal account type.

Example:

```
SBI Bank
  |
  | ₹10,000
  v
Cash
```

This is:

```
TRANSFER
```

The implementation will represent Cash as an external/untracked destination rather than introducing Cash as a fourth formal account type.

No expense is generated.

47. Cash Deposit

Example:

```
Cash
|
| ₹5,000
v
SBI Bank
```

This is:

```
TRANSFER
```

No expense or income is generated.

48. Investment

Investment is represented as:

```
EXPENSE
```

Example:

```
SBI Bank
|
| ₹20,000
v
Investment
```

Transaction:

```
Transaction_Type = EXPENSE
Category = Investment
Payment_Method = BANK
Paid_From_Account_ID = SBI Bank
Amount = 20000
```

The category must exist in the Categories master sheet.

49. Loan Payment

Loan payment is represented as:

EXPENSE

Example:

```
SBI Bank
|
| ₹30,000
v
Loan Payment
```

Transaction:

```
Transaction_Type = EXPENSE
Category = Loan Payment
Payment_Method = BANK
Paid_From_Account_ID = SBI Bank
```

The category is maintained in the Categories master sheet.

50. Balance Calculation

Balances are derived from:

```
Opening Balance
+
Income
+
Incoming Transfers
-
Expenses
-
Outgoing Transfers
```

Only active transactions are included.

Status = ACTIVE

51. Bank Balance

Current Bank Balance =
Opening Balance
+ Income Received
+ Incoming Transfers
- Expenses Paid
- Outgoing Transfers

52. Credit Card Outstanding

Conceptually:

Current Outstanding =
Opening Outstanding
+ Credit Card Expenses
+ Transfers Into Credit Card
- Credit Card Payments
- Transfers Out Of Credit Card

The most important technical rule is:

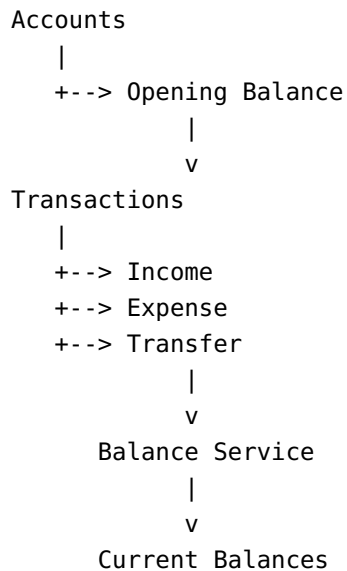
Credit Card Bill Payment
=
TRANSFER
≠
EXPENSE

53. Gift Card Balance

Current Gift Card Balance =
Opening Gift Card Balance
+ Incoming Transfers
- Gift Card Expenses
- Outgoing Transfers

54. Balance Engine

The balance calculation architecture is:



The transaction ledger remains the source of truth.

55. Balance API

Operation:

GET_BALANCES

Example response:

```
{
  "success": true,
  "data": {
    "accounts": [
      {
        "accountId": "ACC001",
        "accountName": "SBI Bank",
        "accountType": "BANK",
        "openingBalance": 100000,
        "currentBalance": 85000
      }
    ]
  }
}
```

```
}  
}
```

56. Frontend Project Structure

Recommended initial structure:

```
finance-tracker/  
|  
+-- index.html  
|  
+-- assets/  
|   |  
|   +-- css/  
|       +-- styles.css  
|   |  
|   +-- js/  
|       +-- app.js  
|       +-- api.js  
|       +-- transactions.js  
|       +-- accounts.js  
|       +-- validation.js  
|       +-- ui.js  
|       +-- utils.js  
|  
+-- components/  
|   +-- transaction-form.js  
|   +-- transaction-list.js  
|   +-- account-summary.js  
|  
+-- config/  
|   +-- environment.js  
|  
+-- README.md
```

The exact structure can be refined during implementation.

57. Frontend Modules

app.js

Responsible for:

- application startup

- loading master data
 - initializing UI
 - registering event handlers
-

api.js

Responsible for:

- API URL
 - HTTP requests
 - JSON parsing
 - common API error handling
-

transactions.js

Responsible for:

- create
 - update
 - delete
 - retrieve
 - transaction-list rendering
-

validation.js

Responsible for:

- form validation
 - amount validation
 - date validation
 - account validation
 - category validation
-

ui.js

Responsible for:

- dynamic form fields
 - loading indicators
 - dialogs
 - success messages
 - error messages
-

utils.js

Responsible for:

- date formatting
 - amount formatting
 - common helper functions
-

58. Google Apps Script Project Structure

Recommended structure:

```
Google Apps Script
|
+-- Code.gs
+-- Api.gs
+-- TransactionService.gs
+-- AccountService.gs
+-- BalanceService.gs
+-- ValidationService.gs
+-- SheetRepository.gs
+-- ResponseUtil.gs
+-- Config.gs
```

59. Backend Module Responsibilities

Code.gs

Entry points:

```
doGet(e)
doPost(e)
```

Api.gs

Responsible for:

- request parsing
 - action routing
 - response generation
-

TransactionService.gs

Responsible for:

```
createTransaction()  
updateTransaction()  
deleteTransaction()  
getTransactions()  
getTransaction()
```

AccountService.gs

Responsible for:

```
getAccounts()  
validateAccount()
```

BalanceService.gs

Responsible for:

```
getBalances()  
calculateAccountBalance()
```

ValidationService.gs

Responsible for:

```
validateTransaction()  
validateExpense()  
validateIncome()  
validateTransfer()  
validateAccount()  
validateCategory()
```

SheetRepository.gs

Responsible for:

- reading sheets
- writing rows
- updating rows
- finding records

Business logic should remain outside the repository layer.

60. Backend Architecture

Recommended logical structure:

```
API Layer
|
v
Service Layer
|
v
Repository Layer
|
v
Google Sheets
```

Example:

```
CREATE_TRANSACTION
|
v
TransactionService
|
v
SheetRepository
|
v
Transactions Sheet
```

61. Application Startup Flow

```
User opens website
|
```



```
      v
Cloudflare Pages
      |
      v
index.html
      |
      v
JavaScript initialization
      |
      v
GET_MASTER_DATA
      |
      v
Google Apps Script
      |
      +----- Accounts Sheet
      |
      +----- Categories Sheet
      |
      +----- Configuration Sheet
      |
      v
JSON response
      |
      v
Frontend application state
      |
      v
Render UI
```

62. Add Expense Flow

```
Open Transaction Form
      |
      v
Select Expense
      |
      v
Enter Date
      |
      v
Enter Amount
      |
      v
Select Category
      |
      v
Select Payment Method
```

```

      |
      v
Select Paid From
      |
      v
Enter Notes
      |
      v
Click Save
      |
      v
Frontend Validation
      |
      v
API Request
      |
      v
Backend Validation
      |
      v
Generate Transaction ID
      |
      v
Write to Transactions Sheet
      |
      v
Return Success
      |
      v
Refresh Transaction List
      |
      v
Refresh Balances

```

63. Add Income Flow

```

Select Income
      |
      v
Enter Date
      |
      v
Enter Amount
      |
      v
Select Category
      |
      v

```

```
Select Received Into
    |
    v
Enter Notes
    |
    v
Save
    |
    v
Validate
    |
    v
API
    |
    v
Google Apps Script
    |
    v
Transactions Sheet
    |
    v
Refresh Balances
```

64. Add Transfer Flow

```
Select Transfer
    |
    v
Enter Date
    |
    v
Enter Amount
    |
    v
Select From
    |
    v
Select To
    |
    v
Enter Notes
    |
    v
Save
    |
    v
Validate
    |
```

```
      v
API
  |
  v
Transactions Sheet
  |
  v
Balance Calculation
  |
  v
Refresh UI
```

65. Edit Transaction Flow

```
Transaction List
  |
  v
Click Edit
  |
  v
Load Transaction
  |
  v
Populate Form
  |
  v
Modify Transaction
  |
  v
Validate
  |
  v
UPDATE_TRANSACTION
  |
  v
Backend Validation
  |
  v
Update Sheet Row
  |
  v
Recalculate Balances
  |
  v
Refresh UI
```

66. Delete Transaction Flow

```
Click Delete
  |
  v
Confirmation
  |
  v
DELETE_TRANSACTION
  |
  v
Find Transaction
  |
  v
Status = DELETED
  |
  v
Recalculate Balances
  |
  v
Refresh UI
```

67. Validation Rules

Common Validation

Date

Required.

Format:

DD-MM-YYYY

Example:

07-09-2026

Historical transactions are supported.

Amount

Required.

Must be:

> 0

Zero and negative values are not allowed.

Notes

Optional.

68. Expense Validation

Required:

Transaction Type
Date
Amount
Category
Payment Method
Paid From

The selected account must match the payment method.

Example:

Payment Method = CREDIT_CARD

Then:

Paid From Account_Type = CREDIT_CARD

69. Income Validation

Required:

Transaction Type
Date
Amount

Category
Received Into

70. Transfer Validation

Required:

Transaction Type
Date
Amount
From
To

Rules:

From != To
Amount > 0
Both accounts must be valid

71. Account Validation

Every account referenced by a transaction must exist in the Accounts master sheet.

Account matching is based on:

Account_ID

72. Category Validation

Every category referenced by an Expense or Income transaction must exist in the Categories master sheet.

Category matching is based on:

Category_ID

73. Date Formatting

The application must consistently display dates as:

```
DD-MM-YYYY
```

Examples:

```
01-01-2026  
15-06-2026  
07-09-2026  
31-12-2026
```

No time should be shown in the business transaction date.

74. Frontend State

The frontend may maintain temporary application state:

```
{  
  accounts: [],  
  categories: [],  
  transactions: [],  
  configuration: {}  
}
```

This state exists only for UI behavior.

Google Sheets remains the persistent source of truth.

75. No IndexedDB

IndexedDB is not required as the primary data store.

The application requires shared data across browsers/devices.

Therefore:

```
Browser  
|  
v
```



```
Google Apps Script
|
v
Google Sheets
```

is used instead of browser-local persistence.

76. Security

The frontend must not contain:

- spreadsheet credentials
- service account credentials
- private API keys
- Google client secrets

The Google Sheet should not be directly accessed by the browser.

The browser should communicate through the Apps Script API.

77. API Error Handling

Recommended error codes:

```
VALIDATION_ERROR
INVALID_TRANSACTION_TYPE
INVALID_AMOUNT
INVALID_DATE
ACCOUNT_NOT_FOUND
CATEGORY_NOT_FOUND
TRANSACTION_NOT_FOUND
SERVER_ERROR
SHEET_ERROR
```

Example:

```
{
  "success": false,
  "error": {
    "code": "ACCOUNT_NOT_FOUND",
    "message": "Selected account does not exist"
  }
}
```

78. Frontend Error Handling

The UI should distinguish between:

Validation Error

```
Amount is required.
```

API Error

```
Unable to save transaction.  
Please try again.
```

Network Error

```
Unable to connect to the server.
```

79. Logging

Google Apps Script should log important technical operations:

```
CREATE_TRANSACTION  
UPDATE_TRANSACTION  
DELETE_TRANSACTION  
GET_TRANSACTIONS  
GET_BALANCES  
API_ERROR  
VALIDATION_ERROR  
SHEET_ERROR
```

Logs should not unnecessarily expose sensitive information.

80. Data Integrity

The following relationships exist:

```
Transactions.Category_ID  
|
```

```
      v
Categories.Category_ID
```

```
Transactions.Paid_From_Account_ID
      |
      v
Accounts.Account_ID
```

```
Transactions.Received_Into_Account_ID
      |
      v
Accounts.Account_ID
```

```
Transactions.From_Account_ID
      |
      v
Accounts.Account_ID
```

```
Transactions.To_Account_ID
      |
      v
Accounts.Account_ID
```

81. Master Data Loading

On application startup:

```
GET_MASTER_DATA
```

returns:

```
Accounts
Categories
Configuration
```

The frontend uses these values to dynamically populate:

- Category dropdown
- Payment Method account dropdown
- Income receiving-account dropdown
- Transfer From dropdown

- Transfer To dropdown

82. Transaction List

Phase 1 should provide a transaction list.

Recommended display:

Date
Type
Category
From / Paid From
To / Received Into
Amount
Notes
Actions

The exact UI layout can be refined later.

83. Transaction Display

Expense:

07-09-2026 | Expense | Food | SBI Bank | ₹2,000

Income:

07-09-2026 | Income | Salary | SBI Bank | ₹100,000

Transfer:

07-09-2026 | Transfer | SBI Bank → HDFC Bank | ₹20,000

84. Search and Filtering

Search and filtering are not required in Phase 1.

They are planned for Phase 2.

Potential future filters:

```
Date Range
Transaction Type
Category
Account
Payment Method
Amount
```

85. Phase 2 Dashboard

Phase 2 may introduce:

```
Dashboard
|
+-- Total Income
+-- Total Expense
+-- Net Cash Flow
+-- Transfers
+-- Bank Balances
+-- Credit Card Outstanding
+-- Gift Card Balances
|
+-- Monthly Expense Chart
+-- Category Expense Chart
+-- Account Expense Chart
+-- Credit Card Spending
+-- Gift Card Spending
+-- Money Flow Visualization
```

Chart.js will be used for charts.

86. Phase 3 Authentication

Phase 1:

```
No Login
No Registration
No User-Level Access
```

Phase 3 may introduce:

```
User
|
v
Authentication
|
v
Authorization
|
v
User-specific Data
```

The exact authentication technology and data model will be defined when Phase 3 is planned.

87. Concurrency

Google Apps Script writes should use appropriate locking where concurrent writes are possible.

Recommended write flow:

```
Acquire Lock
|
v
Validate Request
|
v
Generate ID
|
v
Write Data
|
v
Release Lock
```

This reduces the risk of conflicting simultaneous writes.

88. Performance Considerations

Google Sheets is considered sufficient for the initial Phase 1 implementation.

The implementation should minimize unnecessary spreadsheet operations.

Recommended approach:

```
One master-data read
One transaction operation
One balance calculation
```

where practical.

The implementation should avoid repeatedly reading the same sheet inside a single request.

Caching may be introduced if required during implementation.

89. Backup

The Google Spreadsheet should be backed up periodically.

Initial backup can be handled through Google Sheets/Google Drive capabilities.

Automated backup is not part of the initial application functionality.

90. Development Environment

Recommended development environment:

```
Operating System:
Windows / macOS / Linux

IDE:
Visual Studio Code

Browser:
Google Chrome

Version Control:
Git

Repository:
GitHub
```

91. Git Repository

Recommended structure:

```
finance-tracker/  
|  
+-- frontend/  
|  
+-- backend/  
|  
+-- docs/  
|   +-- BRD.md  
|   +-- TDD.md  
|  
+-- README.md
```

92. API Testing

Postman or Bruno can be used to test:

```
GET_MASTER_DATA  
GET_TRANSACTIONS  
GET_TRANSACTION  
CREATE_TRANSACTION  
UPDATE_TRANSACTION  
DELETE_TRANSACTION  
GET_BALANCES
```

API testing should be completed before full frontend integration.

93. Initial API Testing Sequence

Test 1

```
GET_MASTER_DATA
```

Verify:

```
Accounts  
Categories  
Configuration
```

Test 2

Create Expense.

Test 3

Retrieve Transactions.

Test 4

Create Income.

Test 5

Create Transfer.

Test 6

Update Transaction.

Test 7

Delete Transaction.

Test 8

Retrieve Balances.

94. Critical Test Cases

Bank Expense

Bank → Merchant

Expected:

Expense
Bank balance decreases

Credit Card Expense

Credit Card → Merchant

Expected:

Expense
Credit Card outstanding increases

Credit Card Payment

Bank → Credit Card

Expected:

Transfer
Bank balance decreases
Credit Card outstanding decreases
Expense unchanged

Gift Card Purchase

Credit Card → Gift Card

Expected:

Transfer
Credit Card outstanding increases
Gift Card balance increases
Expense unchanged

Gift Card Usage

Gift Card → Merchant

Expected:

Expense
Gift Card balance decreases

Bank-to-Bank Transfer

Bank → Bank

Expected:

Transfer
Source balance decreases
Destination balance increases
Total expense unchanged

Edit Expense

Original:

₹1,000

Change to:

₹2,000

Expected:

Expense increases by ₹1,000
Source account balance decreases by additional ₹1,000

Delete Expense

Expected:

Expense impact removed
Source account balance restored

95. Implementation Sequence

Step 1 — Create Google Spreadsheet

Create:

```
Accounts  
Categories  
Transactions  
Configuration
```

Step 2 — Configure Accounts

Populate:

```
Account_ID  
Account_Name  
Account_Type  
Opening_Balance
```

Step 3 — Configure Categories

Populate:

```
Category_ID  
Category_Name
```

Step 4 — Create Apps Script

Create the backend project and connect it to the spreadsheet.

Step 5 — Implement Repository Layer

Implement:

```
readAccounts()  
readCategories()  
readTransactions()
```

```
appendTransaction()  
updateTransaction()
```

Step 6 — Implement Validation

Implement:

```
validateTransaction()  
validateAccount()  
validateCategory()  
validateDate()  
validateAmount()
```

Step 7 — Implement Transaction Service

Implement:

```
createTransaction()  
updateTransaction()  
deleteTransaction()  
getTransactions()  
getTransaction()
```

Step 8 — Implement Balance Service

Implement:

```
getBalances()  
calculateAccountBalance()
```

Step 9 — Test Backend APIs

Use:

```
Postman / Bruno
```

Step 10 — Build Frontend

Create:

```
index.html  
CSS  
JavaScript
```

Step 11 — Implement Master Data Loading

Call:

```
GET_MASTER_DATA
```

Step 12 — Implement Dynamic Transaction Form

Implement:

```
Expense  
Income  
Transfer
```

Step 13 — Implement CRUD

Implement:

```
Add  
View  
Edit  
Delete
```

Step 14 — Implement Balance Display

Connect:

```
GET_BALANCES
```

Step 15 — Deploy Frontend

Deploy the static frontend to Cloudflare Pages.

Step 16 — End-to-End Testing

Test complete financial scenarios.

96. Complete Example

Assume:

SBI Bank Opening Balance = ₹100,000

Salary

Salary → SBI Bank
₹100,000

Result:

SBI = ₹200,000
Income = ₹100,000

Hotel Purchase on Credit Card

ICICI Credit Card → Hotel
₹10,000

Result:

Expense = ₹10,000
CC Outstanding = ₹10,000

Credit Card Payment

SBI Bank → ICICI Credit Card
₹10,000

Result:

SBI = ₹190,000
CC Outstanding = ₹0
Expense = ₹10,000

The ₹10,000 payment does not create another expense.

Gift Card Purchase

ICICI Credit Card → Amazon Gift Card
₹5,000

Result:

CC Outstanding = ₹5,000
Gift Card Balance = ₹5,000
Expense = ₹10,000

Gift Card Usage

Amazon Gift Card → Merchant
₹1,000

Result:

Gift Card Balance = ₹4,000
Expense = ₹11,000

97. Core Financial Principle

The technical implementation must preserve:

MOVEMENT OF MONEY $\neq$ EXPENSE

Examples:

Bank $\rightarrow$ Bank
= Transfer

Bank $\rightarrow$ Credit Card
= Transfer

Credit Card $\rightarrow$ Gift Card
= Transfer

Bank $\rightarrow$ Cash
= Transfer

Cash $\rightarrow$ Bank
= Transfer

Bank $\rightarrow$ Merchant
= Expense

Credit Card $\rightarrow$ Merchant
= Expense

Gift Card $\rightarrow$ Merchant
= Expense

98. Technical Design Principles

The implementation should follow these principles:

Single Transaction Ledger

All transactions are stored in one:

Transactions

sheet.

Master-Driven Data

Accounts and categories come from Google Sheets master data.

ID-Based References

Transactions use:

```
Account_ID
Category_ID
```

rather than storing master names as the primary reference.

Backend-Owned Financial Logic

The backend is authoritative for:

- validation
- transaction persistence
- balance calculation

Derived Balances

Balances are calculated from:

```
Opening Balance
+
Transactions
```

rather than manually maintained as independent transaction values.

Single Dynamic Form

One form supports:

```
Expense
Income
Transfer
```

99. Phase 1 Technical Scope

Phase 1 includes:

- ✓ Google Sheets data store
- ✓ Accounts master
- ✓ Categories master
- ✓ Transactions ledger
- ✓ Configuration

- ✓ Expense
- ✓ Income
- ✓ Transfer
- ✓ Dynamic transaction form
- ✓ Add
- ✓ View
- ✓ Edit
- ✓ Delete
- ✓ Balance calculation
- ✓ Credit Card outstanding
- ✓ Gift Card balance
- ✓ Google Apps Script API
- ✓ Frontend
- ✓ Cloudflare Pages deployment
- ✓ Date format DD-MM-YYYY

100. Phase 2 Technical Scope

Phase 2 includes:

Dashboard
Charts
Search
Filtering
Advanced reporting
Money-flow visualization

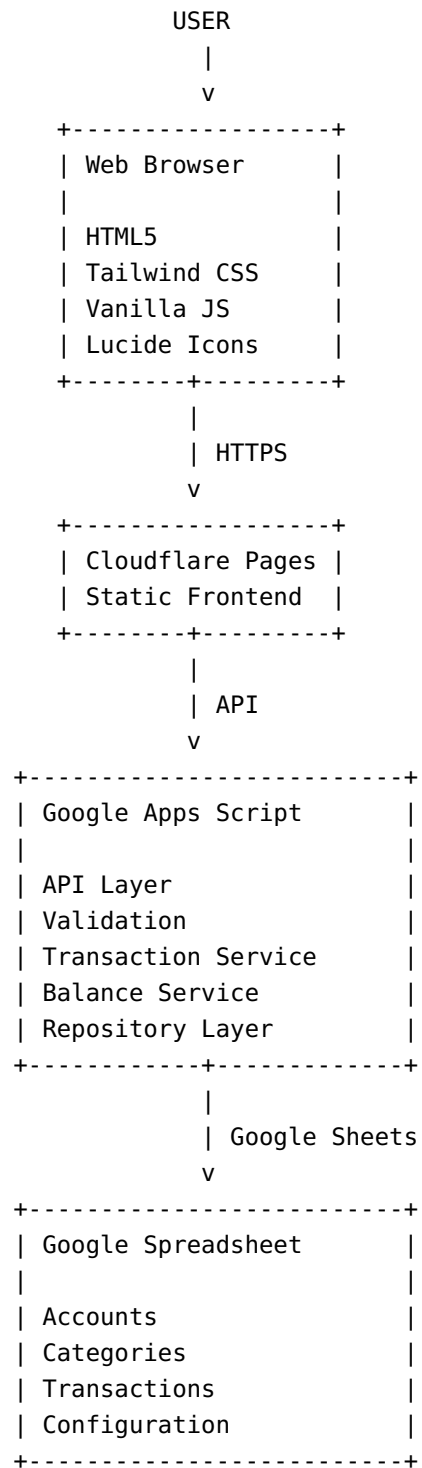
101. Phase 3 Technical Scope

Phase 3 includes:

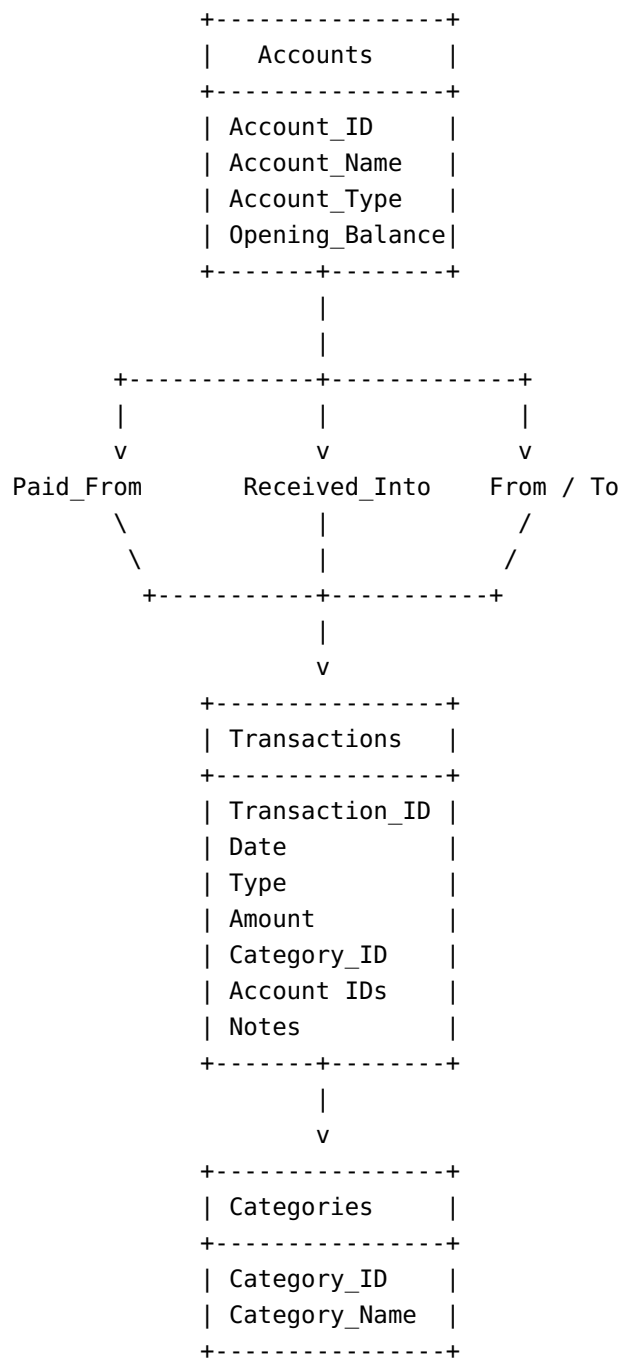
Authentication
Authorization
User management
User-level access
Private data

The technical design for Phase 3 will be defined separately when that phase is initiated.

102. Final Architecture



103. Final Data Architecture



104. Final Mandatory Technical Rules

1. Only three transaction types are supported:
2. Expense

3. Income
4. Transfer
5. Only three formal account types are supported:
6. Bank
7. Credit Card
8. Gift Card
9. Accounts sheet contains only:
10. Account_ID
11. Account_Name
12. Account_Type
13. Opening_Balance
14. Categories sheet contains only:
15. Category_ID
16. Category_Name
17. Transaction dates use DD-MM-YYYY .
18. Credit Card bill payment is a Transfer.
19. Gift Card purchase is a Transfer.
20. Gift Card usage is an Expense.
21. Cash withdrawal/deposit is a Transfer.
22. Investment is an Expense.
23. Loan payment is an Expense.
24. Transfers are not expenses.
25. Transfers are not income.
26. Credit Card bill payments must not be double-counted as expenses.
27. Categories must be loaded dynamically from the Categories sheet.
28. Accounts must be loaded dynamically from the Accounts sheet.

- 29. Account and category names must not be hard-coded into the frontend.
 - 30. Google Sheets is the Phase 1 source of truth.
 - 31. Google Apps Script is responsible for authoritative backend processing.
 - 32. Frontend must not contain private credentials.
 - 33. Balances are derived from opening balances and transactions.
 - 34. Editing a transaction must correctly change its financial impact.
 - 35. Deleting a transaction must correctly remove its financial impact.
 - 36. Phase 1 does not require authentication.
 - 37. Phase 1 does not require search/filter functionality.
 - 38. Phase 1 does not require a dashboard.
 - 39. The technical design will be refined during implementation before being finalized.
-

105. Initial Implementation Readiness

This TDD now provides the initial technical baseline required to begin implementation.

The immediate implementation sequence should be:

```
Google Sheet
  ↓
Master Data
  ↓
Apps Script API
  ↓
Repository Layer
  ↓
Validation
  ↓
Transaction Service
  ↓
Balance Service
  ↓
API Testing
  ↓
Frontend
  ↓
Dynamic Form
```

↓
CRUD
↓
Balance UI
↓
Cloudflare Pages
↓
End-to-End Testing

The design remains open for refinement, particularly around the exact API contract, Google Sheets implementation details, Cash handling, UI structure, and Apps Script implementation patterns.